

Wolfpack Canonical Repository Specification v1

Purpose

This document provides a comprehensive, implementation-ready specification for the canonical repository structure used across all Wolfpack projects. It defines standard directories, file naming conventions, required metadata, and version control policies to ensure consistency, traceability, and governance across development, deployment, and memory archival.

Overview

Each Wolfpack project repository must adhere to a deterministic folder structure with clearly defined purposes. The goal is to make memory, deployment artifacts, decisions, incidents, opportunities, system modules, and operator information machine-readable and human-auditable. The canonical structure also enables automated tooling to ingest, validate, and analyze repository contents.

Directory Layout

- ****/memory/**** – Contains persistent memory files organized by subcategory. Each file is plain markdown with metadata headers.
- ****canon/**** – Core canonical knowledge (e.g., company mission, glossary, high-level principles).
- ****decisions/**** – Architecture Decision Records (ADRs) and significant team decisions. Each file should follow [ADR-001] naming (e.g., ADR-0001-use-postgres.md) and include context, decision, alternatives, and consequences.
- ****deployments/**** – Deployment manifest files for each deployment. Include build info, environment, commit hash, date, approvers, rollback references, and validation results.
- ****incidents/**** – Postmortems for incidents or outages. Include timeline, impact, root cause, remediation, and follow-ups.
- ****opportunities/**** – Records of new opportunities, experiments, and features. Include hypothesis, market signals, sponsors, and outcome.
- ****operators/**** – Team members, roles, permissions, and contact details. Include on-call rotations and responsibilities.
- ****customers/**** – Summaries of customer profiles, contracts, preferences, and interactions. Comply with privacy policies.
- ****systems/**** – High-level system component descriptions, dependencies, constraints, and status.
- ****summaries/**** – Auto-generated summary snapshots of memory for efficient retrieval. Each summary references original files.
- ****/src/**** – The source code of the application or service. Organized by domain modules following DDD (Domain-Driven Design). Include tests alongside code.
- ****/scripts/**** – Utility scripts for maintenance, migrations, and operational tasks. Use descriptive names and include usage documentation in the header.
- ****/tools/**** – Custom build tools, linters, or CLI helpers. All tools should be versioned and documented.

- `**/docs/**` – Human-readable documentation such as API specs, runbooks, and readme files. Auto-generated docs should be in separate subfolders.

- `**/config/**` – Configuration templates and environment-specific overrides. Avoid committing secrets. Use YAML or JSON for structured config.

Naming Conventions

- All markdown files in `**/memory/**` must begin with a YAML front matter block containing metadata: ``title`, `date`, `author`, `tags`, `version`, and `provenance`` (source or system that created it).
- Use ISO 8601 for dates (``YYYY-MM-DD``).
- For ADR filenames, use incremental numbers padded to 4 digits, short slug summary, and ``.md`` extension.
- Deployment manifests named as ``deploy-YYYYMMDD-HHMMSS.md`` to guarantee chronological sorting.
- Incident postmortems named as ``inc-YYYYMMDD-HHMM.md``.

Version Control Policy

- Use Git as the VCS. All changes must be committed with descriptive messages referencing associated tickets or ADRs.
- Require code review with at least one approval before merging to the main branch.
- Enable branch protection rules: enforce CI checks, prevent force-pushes, and restrict direct commits to main.
- Tag releases with semantic versioning (``vX.Y.Z``), linking deployment manifests.

Metadata and Provenance

- Each memory file must include ``provenance`` metadata indicating the source (human, tool, external system) and any transformation steps.
- Maintain a changelog in each file's front matter listing significant edits with timestamps and authors.
- Automated summarization should include a pointer to the source file and summary algorithm version.

Validation and Automation

- Implement a CI job to validate repository structure on each pull request. This job should:
 - Check presence of required directories and files.
 - Validate metadata front matter for required fields and date formats.
 - Enforce naming conventions.
 - Lint ADRs for proper structure.
- Provide CLI tools to scaffold new ADRs, incidents, deployments, etc., ensuring consistent formatting.

Access Control and Security

- Restrict access to `**/customers/**` based on need-to-know. Sensitive fields must be encrypted or stored in secure secrets management rather than plaintext.
- Employ GitHub branch permissions to limit modifications on memory content to authorized roles.
- Maintain audit logs for all changes to memory directories.

Extensibility

- New memory categories should be proposed via ADR and added to the `memory/README.md` with purpose and expected contents.
- The repository may include additional top-level directories (e.g., `/terraform/`, `/services/`) as needed but must maintain the core memory structure.